Magic: The Gathering Multiplayer Network Protocol

A required-scope MTGNP v1.0 implementation in Python: one authoritative server, exactly two TCP clients, desktop and terminal controls, the supplied fixed card catalog, mulligans, turns, priority and stack handling, combat, reconnection, and game restart. Both clients reuse the same transport, state store, and controller layers.

Build and setup

- Python 3.12 or newer
- Python's optional Tcl/Tk component for the desktop client
- No third-party runtime packages

No compilation or package installation is required. Clone or extract the repository,

open PowerShell in the repository root, and verify the source before running it:

```
python --version
python -m compileall -q common server client scripts
```

The first command should report Python 3.12 or newer. To verify that the optional

desktop GUI is available, run `python -m tkinter`; a small Tk window should open.

The terminal client can still be used when Tcl/Tk is unavailable.

Start the server and two desktop clients

Open three terminals:

```
python -m server.main --host 127.0.0.1 --port 4444
python -m client.gui_main --host 127.0.0.1 --port 4444 --id player_1 --deck decks/red.json
python -m client.gui_main --host 127.0.0.1 --port 4444 --id player_2 --deck decks/black.json
```

For the required verbose demonstration, add `--verbose` (or `-v`) to all three

commands:

```
python -m server.main --host 127.0.0.1 --port 4444 --verbose
python -m client.gui_main --host 127.0.0.1 --port 4444 --id player_1 --deck decks/red.json --verbose
python -m client.gui_main --host 127.0.0.1 --port 4444 --id player_2 --deck decks/black.json --verbose
```

Verbose mode is disabled by default. When enabled at startup, each process prints

every complete PDU it sends and receives as readable JSON with a clearly labelled

`SEND` or `RECV` prefix. Restart without the flag to disable it.

The connection options are optional for the GUI. Running `python -m client.gui_main` opens a form for the host, port, player ID, and deck file. Each client is a separate process, so the server still enforces exactly two simultaneous player connections.

Select cards through native Tk buttons that display cached, transparent rounded-card sprites. The hand shows four large overlapping card positions at a time and scrolls directly with the mouse wheel. Hovering briefly lifts and enlarges a card; clicking keeps the larger selected state and its own card-color outline. Battlefield rows use compact previews so the full match surface stays visible without an outer window scrollbar. Card-color outlines remain visible for white, blue, black, red, green, colorless, and multicolor cards. The card image itself carries the name, cost, type, rules, and combat values, so the desktop client does not duplicate that metadata in captions or a separate details panel.

Each player's public life, hand, library, graveyard, and exile counts sit directly above that player's battlefield. Your strip also shows land-play status, permanent count, and colored `Potential mana` pips. Potential mana is not a client-side mana pool: it is a display-only count of the untapped Mountain, Island, Forest, Swamp, Plains, and Sol Ring sources

the server currently recognizes.

The top phase strip shows the previous phase, an arrow, the current phase, another arrow, and the next phase. Available actions and activity remain beside the board, and all three card rows support wheel navigation without visible scrollbars. If the transport fails, the client closes the failed session and restores the connection form with the previous host, port, player ID, and deck values.

Keyboard navigation uses `Tab` to move focus and `Enter` or `Space` to select a focused card. `Escape` cancels dialogs, `Return` confirms target, blocker, and damage-order prompts, and `Up`/`Down` reorder combat damage. Structured actions such as spell targets, blockers, and damage order open focused prompts.

Terminal client

Open three terminals:

```
python -m server.main --host 127.0.0.1 --port 4444
python -m client.main --host 127.0.0.1 --port 4444 --id player_1 --deck decks/red.json
python -m client.main --host 127.0.0.1 --port 4444 --id player_2 --deck decks/black.json
```

The same `--verbose` or `-v` flag works on both terminal clients. The server also

accepts `--reconnect-timeout SECONDS`.

Run the bounded real-socket smoke demo with:

```
python -m scripts.demo_game
```

Terminal commands

Command	Purpose
`ready CARD_ID...`	Submit a deck manually
`keep [CARD_ID...]`	Keep the mulligan hand and bottom the required cards
`mulligan`	Reroll the current opening hand
`land CARD_ID`	Play one land during an active-player main phase
`cast CARD TARGET R=1 X=2`	Cast with color counts; use `-` for no target
`activate SOURCE INDEX TARGET... COST_SOURCE...`	Submit an activated-ability request
`pass`	Pass the current priority token
`attack CARD_ID...`	Declare attackers
`block ATTACKER BLOCKER...`	Declare blockers for one attacker
`damage-order ATTACKER BLOCKER...`	Order multiple blockers
`trigger-order TRIGGER_ID...`	Respond to a trigger-order request
`trigger-choice ID yes/no [TARGET]`	Respond to an optional/targeted trigger request
`discard CARD_ID...`	Discard to seven during cleanup
`concede`	Concede the current game
`state`, `help`, `quit`	Inspect the last snapshot, show help, or exit

The server checks legality. A rejected action produces `ERROR` and does not intentionally update the game state. Clients render only server messages and never calculate outcomes.

Protocol and architecture

Every message is compact UTF-8 JSON prefixed by a four-byte unsigned big-endian payload length. Payloads are capped at 65,535 bytes. `common.pdus` defines all 25 supplied PDU names, their direction, required fields, and all 12 specified error codes.

Each accepted socket has a reader thread. Readers only decode frames and place seat-tagged events on a shared queue. One `GameServer` loop consumes that queue and is the only owner of mutable `GameEngine` state. Per-seat locks serialize outgoing frames. A third simultaneous connection receives an error and is closed.

The server sends personalized snapshots: a player sees their own hand, the opponent hand count, library counts, and all public zones. Neither library order nor the opponent hand is sent. After game over, sockets remain available and both players can submit fresh `PLAYER_READY` messages.

The client boundary keeps presentation separate from protocol behavior:

- `ClientNetwork`: TCP, framing, receive thread, heartbeat hooks
- `ClientStateStore`: authoritative snapshot replacement and subscriptions
- `ClientController`: presentation-neutral action methods
- `GameApplication`: Tkinter presentation and GUI-thread event bridge
- `TerminalUI`: command parsing and rendering only

Both presentations subscribe to `ClientStateStore` and call `ClientController`; neither imports server rules.

Implemented rules

- Lobby setup, unique IDs, supplied-instance deck validation (1–50 cards), server shuffle, 20 life, and random first player
- London-style reroll mulligans, including short-deck handling and card-bottom validation
- Required phase cycle from Untap through Cleanup; Untap and Cleanup are automatic and priority-free, and the first player skips the first-turn draw
- One land per turn, declared color-count payment, atomic server selection/tapping of mana sources, untap and temporary-effect cleanup
- Server-granted request tokens, active/non-active priority transfer, timeout-as-pass enforcement, two-pass advancement, and LIFO stack resolution
- Legal targets are checked when cast and again at resolution; invalidated spells fizzle
- Attackers, blockers, multiple-blocker damage order, first strike/double-strike engine support, simultaneous damage, lethal state checks, and no trample carry-over
- `LIFE_ZERO`, `DECK_EMPTY`, `CONCEDE`, and `DISCONNECT` game-over reasons
- Mandatory Gray Merchant enter trigger on the stack

The five required card effects are Lightning Bolt, Counterspell, Unsummon, Giant Growth, and Gray Merchant of Asphodel. The finished engine additionally supports Shock, Lava Spike, Rift Bolt, Ponder, Rampant Growth, Dark Ritual, Doom Blade, Terror, Mind Rot, and Raise Dead. All supplied lands provide mana; supplied creatures and permanents retain their listed base statistics and core combat keywords. Other instant, sorcery, triggered, or activated text is intentionally unavailable because complete card-effect coverage is bonus scope.

Supplied card data and artwork

`cards.json` contains the 58 definitions from `mtgnp_master_card_list.pdf`. `CardCatalog` derives exactly 312 protocol instance IDs in `<base_id>_<three-digit copy>` form. No outside card database or rule source is used.

The desktop client includes one optimized full-card image for each supplied card under `client/assets/cards/`. These images were downloaded through Scryfall for display only; `manifest.json` records the Scryfall ID, source page, image URL, and credited artist. The rules engine never reads Scryfall data and the client makes no runtime web requests. See `THIRD_PARTY_NOTICES.md` for the required unofficial fan-content notice.

Test

```
python -m compileall -q common server client tests scripts
python -m unittest discover -s tests -v
```

The suite covers framing, every PDU contract, catalog totals, hidden information, lifecycle and mulligans, priority/stack behavior, five effects, phases/combat, two-seat connections, GUI projections/actions/assets, Tk rendering, and real-loopback two-client behavior.

Known limitations and RFC deviations

The following limits are intentional and known at submission time:

- Fifteen named effects are implemented, including all five required effects and ten additional spells. Other rules text in the supplied catalog is unavailable; complete card-effect coverage is bonus scope.
- The schemas and terminal commands expose activated-ability and trigger-choice PDUs, but the supported card subset does not implement a general interpreter for arbitrary activated, optional, or simultaneous triggered abilities.
- Bonus mechanics such as trample carry-over are not implemented. A blocked attacker assigns damage to its blockers and does not deal excess damage to the defending player.
- Authentication, TLS, spectators, matchmaking, persistence, and best-of-three matches are not provided. These are either explicitly outside MTGNP 1.0 or identified by the RFC as deployment concerns rather than required baseline work.
- GUI behavior still requires a final two-machine/manual demonstration on a Python 3.12 installation with Tcl/Tk, including reconnection, stale-action recovery, verbose logging, a complete match, and starting a second game on retained sockets.

Where the RFC text is ambiguous or its examples conflict, the implementation uses these documented interpretations:

- Formal Section 10 field names and normative prose take precedence over conflicting appendix examples.
- The setup transition is `MULLIGAN -> UNTAP`; the first player does not draw on turn one.
- Opening hands draw up to the cards available. A short-deck mulligan bottoms no more cards than the hand contains.
- Reconnection reuses the existing `PLAYER_READY` PDU with the same player ID because the supplied protocol defines no separate reconnect PDU.
- An attacker blocked by multiple creatures assigns damage in submitted order up to remaining lethal toughness; this required baseline does not carry excess damage to the defending player.

- Combat damage remains marked during End of Combat, is cleared when that step closes, and is cleared again idempotently during Cleanup.

Work Distribution Matrix

P means Primary developer, A means Assisting developer, and QA means quality assurance. A task may have more than one primary developer when ownership was shared. All members share final review responsibility and must be able to explain the complete submission.

Task / feature	Elkan Ainer M. La Madrid	Duncan Joseph B. Marcaida	Jerry King H. Deveza	Jean Rondel R. Ponce
TCP server: connection handling, framing, dispatch	P	A	QA	A
Game lifecycle: Lobby, setup, mulligan	A	P	A	A
Turn and phase engine	A	P	A	A
Priority, stack, and spell resolution	A	P	QA	A
Combat system	P	A	A	A
Desktop/terminal clients and state rendering	P	A	A	QA
PDU serialization/deserialization (all 25 types)	A	QA	A	P
Error handling, heartbeat, and disconnect/reconnect	A	QA	QA	P
Verbose PDU logging on client and server	A	QA	A	P
Automated testing and interoperability checks	A	A	P	A
README, demo guide, and AI disclosure	A	A	P	A
Card catalog, required effects, and GUI bonus integration	P	A	P	A

The matrix summarizes the team's implementation and review work plus the repository commit history; it does not replace each member's duty to understand and demonstrate every component.

AI Usage

Tool	How it was used	Human verification
Claude	Helped interpret the supplied RFC and prepare an implementation/delegation proposal.	The team compared suggestions with the supplied RFC before adopting them.
OpenAI Codex	Assisted with repository inspection, design iteration, Python implementation, debugging, test creation, GUI refinement, and documentation.	The team reviewed changes against the RFC, ran the automated suite, and retained final responsibility for the code.

No AI output is treated as a protocol authority. Rules and protocol behavior were checked against the supplied course PDFs and executable tests. Scryfall was used only to verify the 58 supplied card names and acquire bundled display artwork; no external Magic rules or card behavior were imported. Its source URLs and artist credits are recorded in `client/assets/cards/manifest.json` and `THIRD_PARTY_NOTICES.md`.